

# ADVANCED SOFTWARE ENGINEERING

Abdulfattah Alfarra

Development Strategy

# OUTLINE

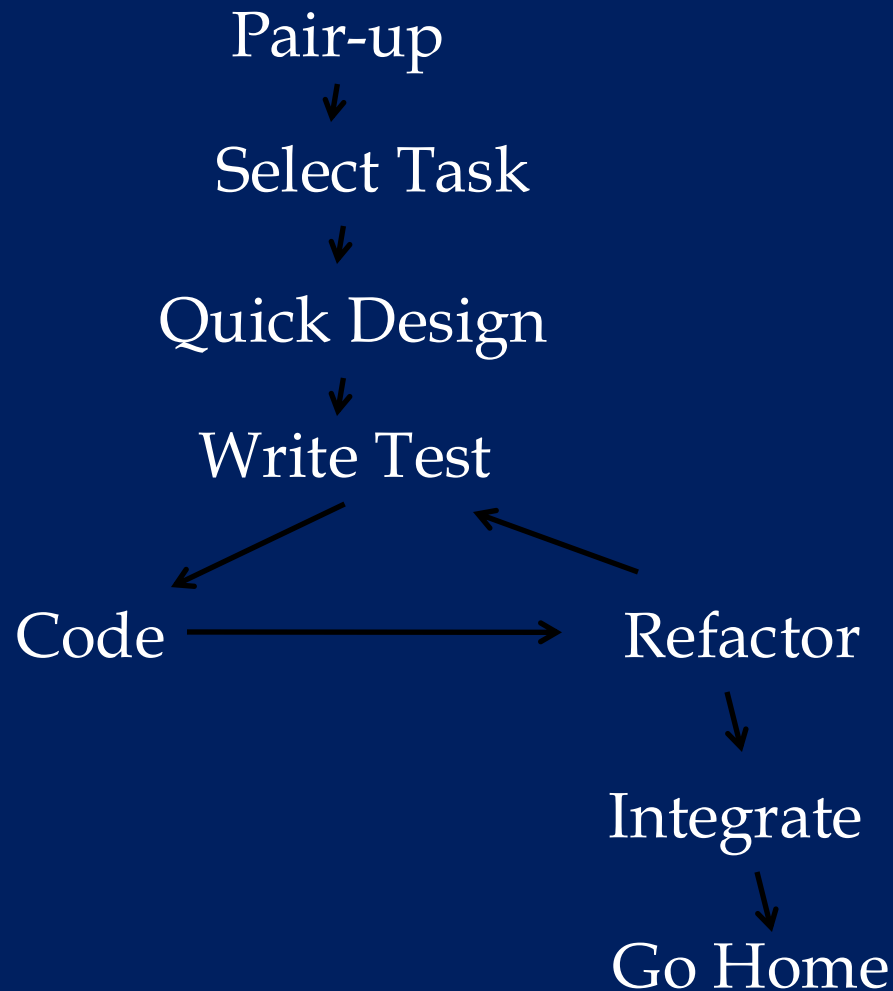
- Definition
- Pair programming
- Collective ownership
- Code standard
- 40 hours week
- Refactoring

# Definition

- ▣ Code is at the heart of any software.  
Better the code, better is the reaction to input.
- ▣ Extreme Programming nominates coding as the **key activity**.
- ▣ The programmer is the heart of XP.
- ▣ In XP, you code because if you don't code, at the end of the day you haven't done anything.

# Definition

An XP developer's daily routine



# Definition

## Pair-up

Developers work in pairs, sharing each workstation. Your pair might already be established if you're still working on a task, otherwise you'll need to find another developer from the team to pair-up with.

## Select Task

Assuming, you're not still working on a task, your pair will need to select the next engineering task to work on.

# Definition

## Quick Design

Design at this level means "how are we going to solve this task?"

## Write Test

Begin the task by writing a test case and check that this test fails before coding. Expand your test to cover everything that could possibly break. Seek clarity from the customer team where required.

# Definition

## Code

Write the code to complete the task. The goal is to write only enough code to ensure the component passes your test case

## Refactor

Revisiting the code, look for areas of simplification and improvement. Stop refactoring when simplicity has been achieved.

# Definition

## Integrate

Add the complete component to the integration or build machine. Verify that the integrated class passes tests. If integration problems continue, discard the code and start again tomorrow.

## Go Home

In keeping with the XP practice of 40-hour weeks, go home at 5 p.m.

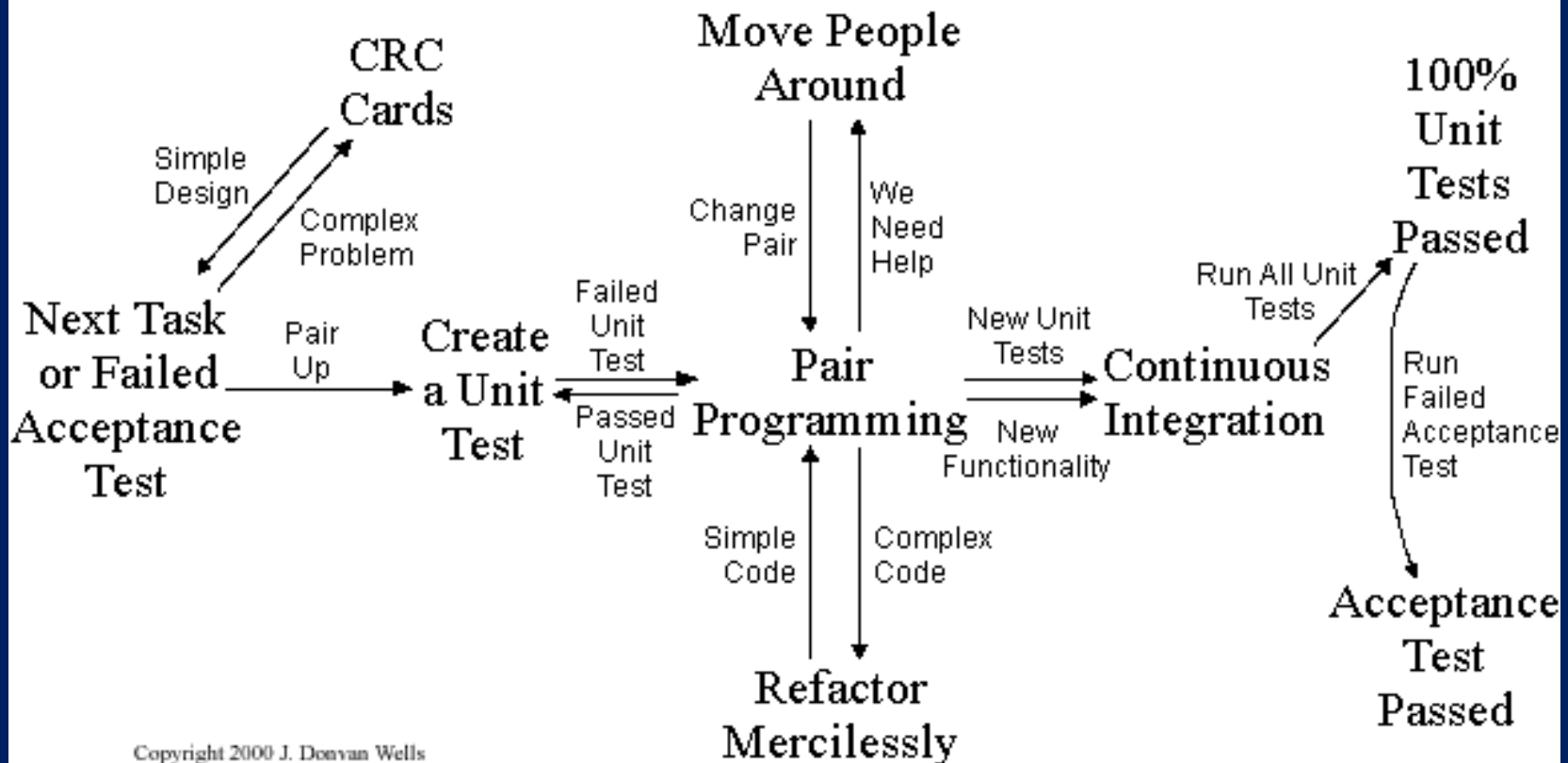


# XP Project Coding



## Collective Code Ownership

Zoom Out



Copyright 2000 J. Donovan Wells

# Pair Programming

- ▣ Two programmers collaborate on the same design, algorithm, code or test case
- ▣ Pairing is dynamic
- ▣ Encourages communication, productivity and enhances code quality
- ▣ Pairing is useful for cross-training established employees and for training new employees



# Pair Programming

## Rules:

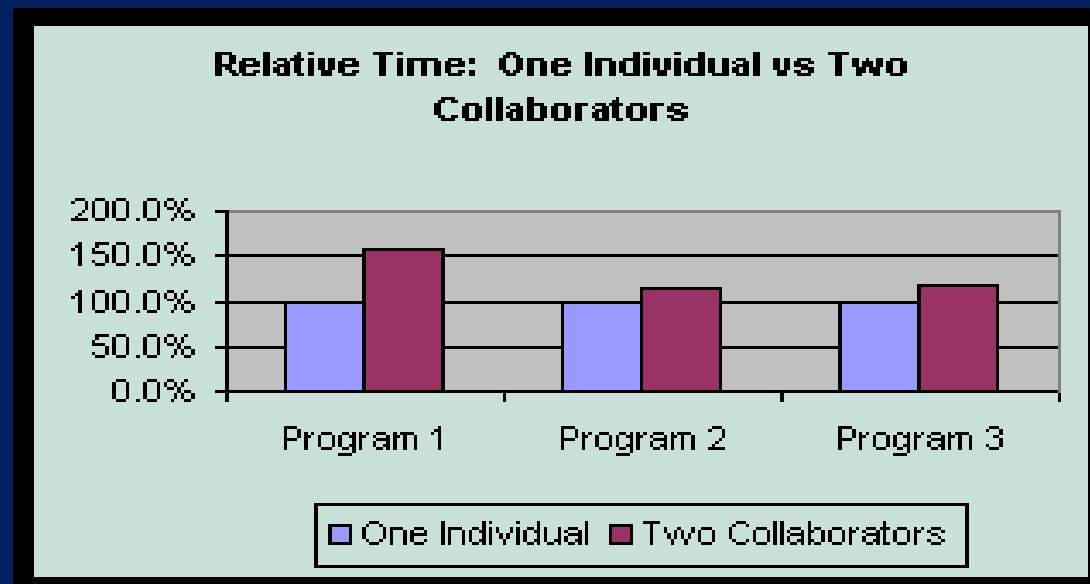
- ▣ All programming sessions are “shoulder to shoulder”: two people at one screen
- ▣ Both must understand and agree with every line of code.
- ▣ Switch drivers from time to time.

# Pair Programming

- ▣ **Pair programming research reveals that:**
  - Pairs use no more man-hours than singles
  - Pairs create fewer defects
  - Pairs create fewer lines of code
  - Pairs enjoy their work more

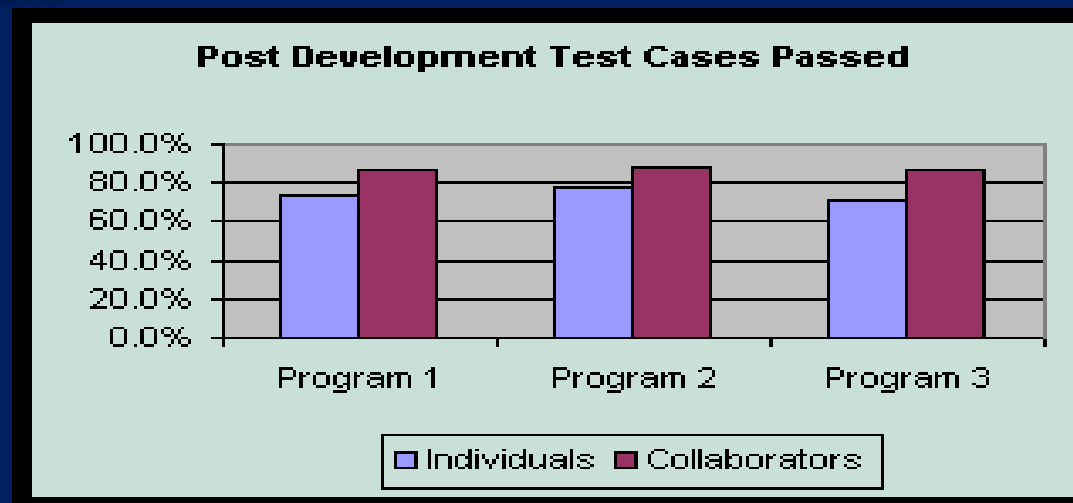
# Pair Programming

University of Utah Experiment: Pairs spent 15% more time on the program than individuals



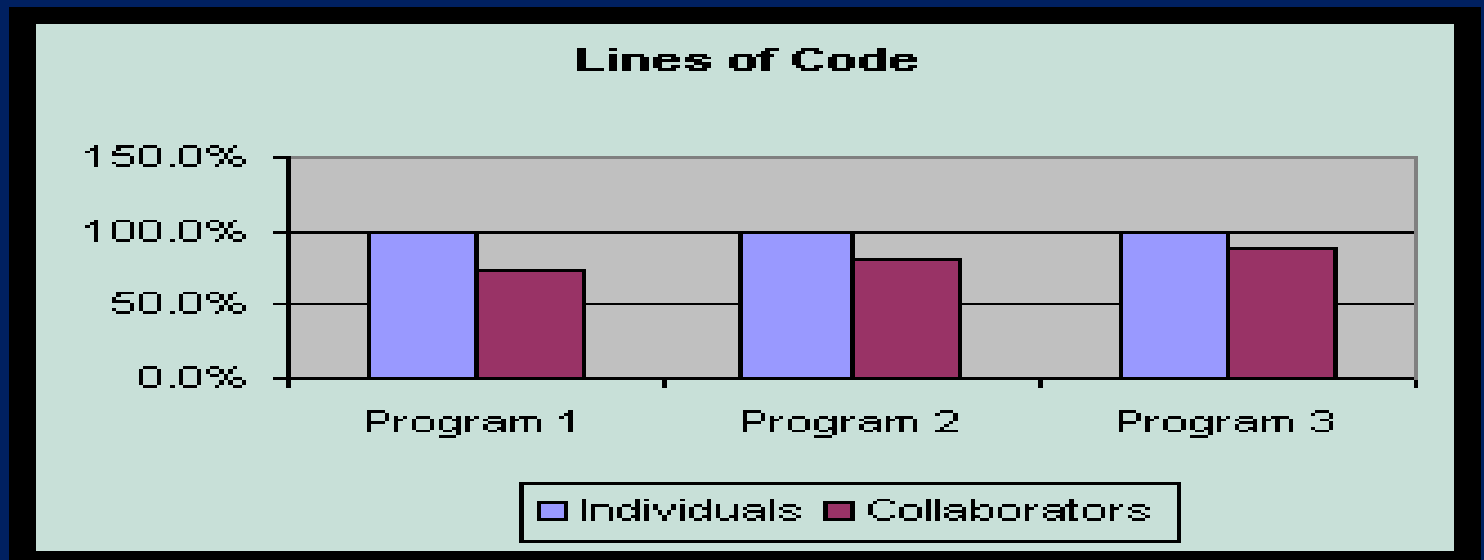
# Pair Programming

University of Utah Experiment: Code written by pairs passed more test cases than code written by individuals



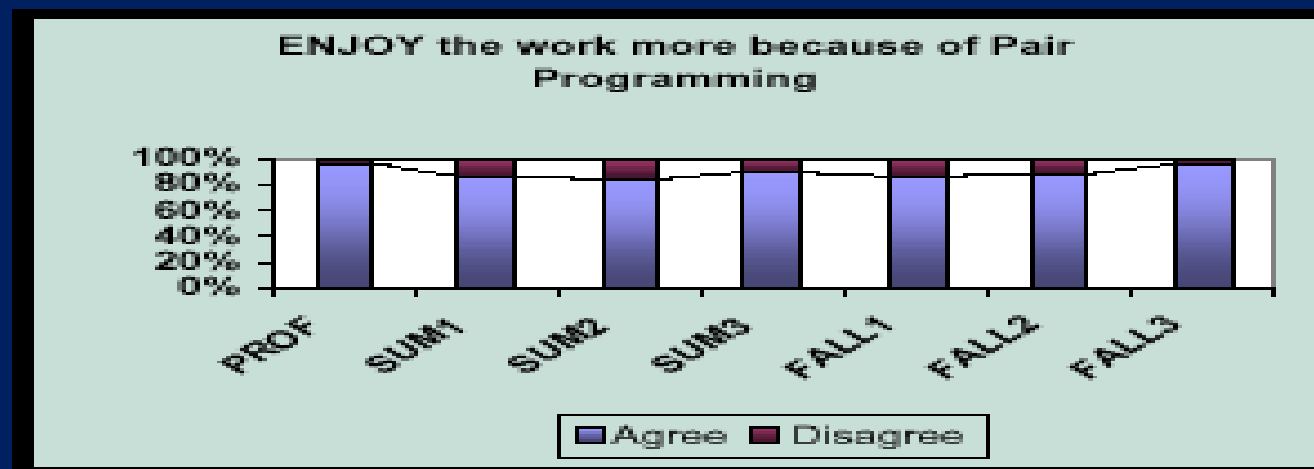
# Pair Programming

University of Utah Experiment: Pairs consistently implemented the same functionality produced by individuals in fewer lines of code



# Pair Programming

University of Utah Experiment: Learning how to program in an environment where there are rapidly tangible results is fun and allows one to learn faster





# Pair Programming

## University of Utah Experiment: Conclusions

The significant benefits of pair programming are that:

- ▣ Many mistakes get caught as they are being typed in rather than in testing.
- ▣ The end defect content is statistically lower
- ▣ The designs are better
- ▣ Code length shorter
- ▣ The team solves problems faster

# Pair Programming

- ▣ Disadvantages of Pair programming
- ▣ Skill Difference: This is the number one potential problem. If the partners are of completely different skill levels, you might have one programmer doing all the work or constantly tutoring the other. This is ok if you want to set up a teacher-student relationship or are introducing a new programmer to your system, but if not, it can defeat the entire purpose of pair-programming.

# Pair Programming

- ▣ **Not actually getting the work done:** For some people pair programming sessions can easily generate in socializing sessions. There are some people who don't work when there is someone next to them examining their work, these people will not benefit much either.



# Pair Programming

- ▣ **Developer egos:** This is something that is not likely to happen in a classroom setting, but in more experienced teams, each programmer might try to push their own ideas of how things should be done (both of which may be perfectly valid). These sort of conflicts can be downright disastrous.

# Pair Programming

- ▣ **Start up Time:** Some start up time needed to adjust to paired environment.

# Pair Programming

- ▣ **Evaluating programmers:** it becomes incredibly difficult to evaluate individuals and to pay them according to their contributions. In a course context, this problem can be overcome in many ways. In the industrial world, this is a true problem without an obvious solution.

# Pair Programming

Many people have been surprised by pair programming. They gave it a try, expecting to hate it, and found they really liked it.

# Collective Ownership

- ▣ What happens when a critical person goes on holiday, stays home with a sick child, takes a new job, or suddenly retires? How much time will you spend training a replacement?



# Collective Ownership

- ▣ *Collective code ownership* spreads responsibility for maintaining the code to all the programmers. Collective code ownership is exactly what it sounds like: everyone shares responsibility for the quality of the code. No single person claims ownership over any part of the system, and anyone can make any necessary changes anywhere.

# Collective Ownership

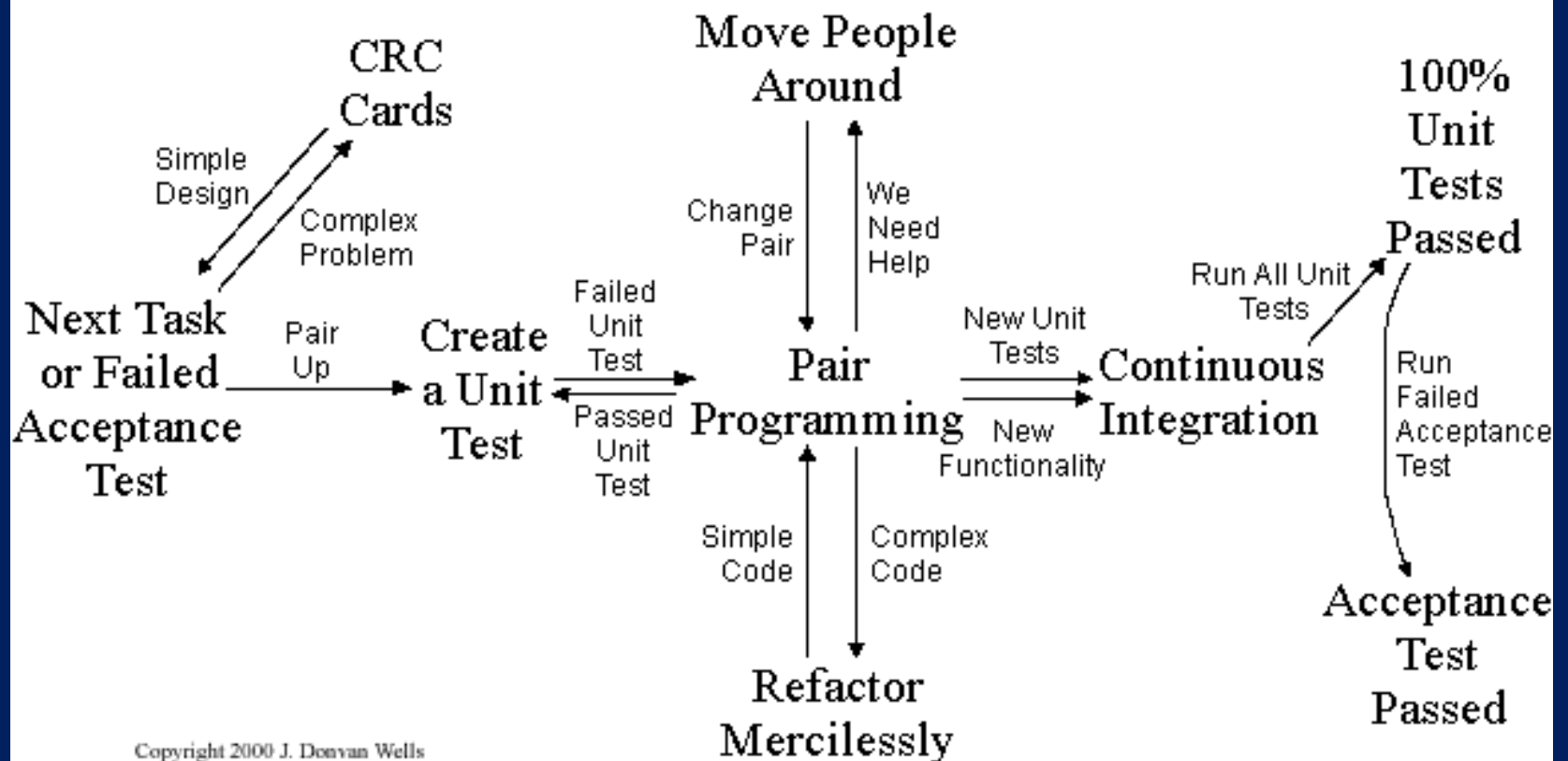
- ▣ Any team member may add to the code at any time
- ▣ Everybody takes responsibility for the whole system
- ▣ Encourages simplicity:
  - Prevents complex code from entering the system
- ▣ Increases individual responsibility and personal power
- ▣ Reduces project risk:
  - Spreads knowledge of the system around the team

# XP Project Coding



## Collective Code Ownership

Zoom Out



Copyright 2000 J. Donovan Wells

# Collective Ownership

## Making Collective Ownership Work

Collective code ownership requires letting go of a little bit of ego:

- ▣ Rather than taking pride in *your code*, take pride in *your team's code*.
- ▣ Rather than complaining when someone edits your code, enjoy how the code improves when you're not working on it.

# Collective Ownership

- ▣ Rather than pushing your personal design vision, discuss design possibilities with the other programmers and agree on a shared solution.

# Collective Ownership

## Benefits

- ▣ **Software development teams are highly unstable:** For XP software teams seem to be ever changing. People constantly coming on board, leaving the team, being sick or run over by a truck. You simply can't rely on a person to be there for a longer period of time to assign him responsibility for any part of a project.
- ▣ **Shared knowledge of the code:** allows programmers to become familiar with more of the code and benefit from the experience of others.

# Collective Ownership

- ▣ **Simpler code:** causes complex code to be found and refactored more quickly as many pairs of eyes read the same code.
- ▣ **Get things done quickly:** removes hurdles so changes can be made by those that need them when they need them.

# Collective Ownership

- ▣ **Improved code quality.** Collective ownership allows everyone to fix problems they find. If you encounter duplication, unclear names, or even poorly designed code, it doesn't matter who wrote it. It's *your* code. Fix it!



# Collective Ownership

## Collective Ownership Drawbacks

- ▣ There is a danger of one person in the office becoming the “bug fix person”. Not many developers enjoy fixing bugs all day, so in my view it would be important to try to make sure that responsibility for fixing bugs is shared equally between the development team

# Collective Ownership

- ▣ If more people are changing any one area of the code base, direct coding conflicts (conflicting change sets) are more likely to occur.
- ▣ People who were responsible for the initial creation of a particular module are often unwilling to let it go and be modified by other developers.

# Coding Standards

- ▣ A coding standards document tells developers how they must write their code. Instead of each developer coding in their own preferred style, they will write all code to the standards outlined in the document.
- ▣ This makes sure that a large project is coded in a consistent style — parts are not written differently by different programmers. Not only does this solution make the code easier to understand, it also ensures that any developer who looks at the code will know what to expect throughout the entire application.

# Coding Standards

- ▣ If you're using XP you will be developing in pairs, each taking turns and at the keyboard. Before coding starts, you will need an agreed upon set of rules on how to attack development. This removes petty arguments about formatting, naming, and such. Without coding standards the swapping of pairs would become difficult because you'd be forced to learn every other developers style and approach.
- ▣ So, coding standards are not just a best practice — they grease the wheels of the development team and ensure that everybody wins.

# Coding Standards

- ▣ The standard should call for the least amount of work possible, consistent with the Once and Only Once rule (no duplicate code). The standard should emphasize communication. Finally, the standard must be adopted voluntarily by the whole team.

# Coding Standards

- ▣ Coding standards are guidelines for code style and documentation.
- ▣ The dream is that any developer familiar with the guidelines can work on any code that followed them.
- ▣ Standards range from a simple series of statements to involved documents.
- ▣ Coding should be done with an aim toward maintenance.

# Coding Standards

- ▣ Decide which standards you apply

Complete Standard

<http://www.oracle.com/technetwork/java/codeconvtoc-136057.html>

# 40-hour week

- ▣ In XP, development is high in communication and has impressive speed. Developers are working in an environment where the stress of change is ever present.
- ▣ Working on XP projects means consistently driving quality and performance throughout the life of the project.



# 40-hour week

- ▣ How do you maintain quality with overtime-heavy teams? **The answer is you can't.**
- ▣ Remembering that an XP core principle is that quality of work can never be compromised, we're going to have to control how many hours developers clock and still maintain high standards.
- ▣ If energy could be directed exclusively toward the project for eight hours per day, ignoring anything outside of the project, **40-hour work** weeks could be successful.

# 40-hour week

- ▣ The idea of the 40-hour week is that team members should work the hours that they can *sustain* quality. Sustainable workload could be 40 hours or around that.

# 40-hour week

- ▣ Establishing a policy of sustainable hours also removes the "black market" in project time. This refers to the very dubious practice of price-fixing the contract (effectively locking the team into charging no more than eight hours per day) while still allowing management to lean on the team to work extended, unpaid hours. For those employees paid on a salary, they are losing personal time for no reward.

# 40-hour week

- ▣ The XP rule is simple — you can't work a second week of overtime. For one week, fine, crank and put in some extra hours. If you come in Monday and say, "To meet our goals, we'll have to work late again," then you already have a problem that can't be solved by working more hours.

# Refactoring

## What it Refactoring?

Refactoring is the technique of improving code without changing functionality. Refactoring is an ongoing process of simplification that applies to code, design, testing, and XP itself.

# Refactoring

- ▣ Here is a simple refactoring. Suppose you have this code:

```
public class Person {  
    private String firstName;  
    public void setFirst(String n) {  
        this.firstName = n; }  
}
```

# Refactoring

- ▣ This code can be improved by picking a better argument name, changing n to firstName:

```
public class Person {  
    private String firstName;  
    public void setFirst(String firstName) {  
        this.firstName = firstName;  
    }  
}
```

# Refactoring

- ▣ The code can be improved even further by renaming the method to `setFirstName()`:

```
public class Person {  
    private String firstName;  
    public void setFirstName(String firstName) {  
        this.firstName = firstName; }  
}
```



# Refactoring

Why should refactoring be done?

1) **Refactoring Improves the Design** of Software. -

Without refactoring, the design of the program will decay. As people change code - changes to realize short-term goals or changes made without a full comprehension of the design of the code - the code loses its structure.

# Refactoring

- 2) Refactoring makes **software easier to understand**. - There are users of your code. The computer, the writer, and the updater. The most important is the updater. Who cares if the compiler takes a few more cycles to compile your code. If it takes someone 3 weeks to update your code that is a problem.

# Refactoring

- 3) Refactoring **helps you find bugs.** - Part of refactoring code is understanding the code and putting that understanding back into the code. In that process a clarification takes place. In that clarification bugs will be found.

# Refactoring

**4) Refactoring Helps You Program Faster:** A good design is essential to maintaining speed in software development. Refactoring helps you develop software more rapidly, because it stops the design of the system from decaying. It can even improve a design.

# Refactoring

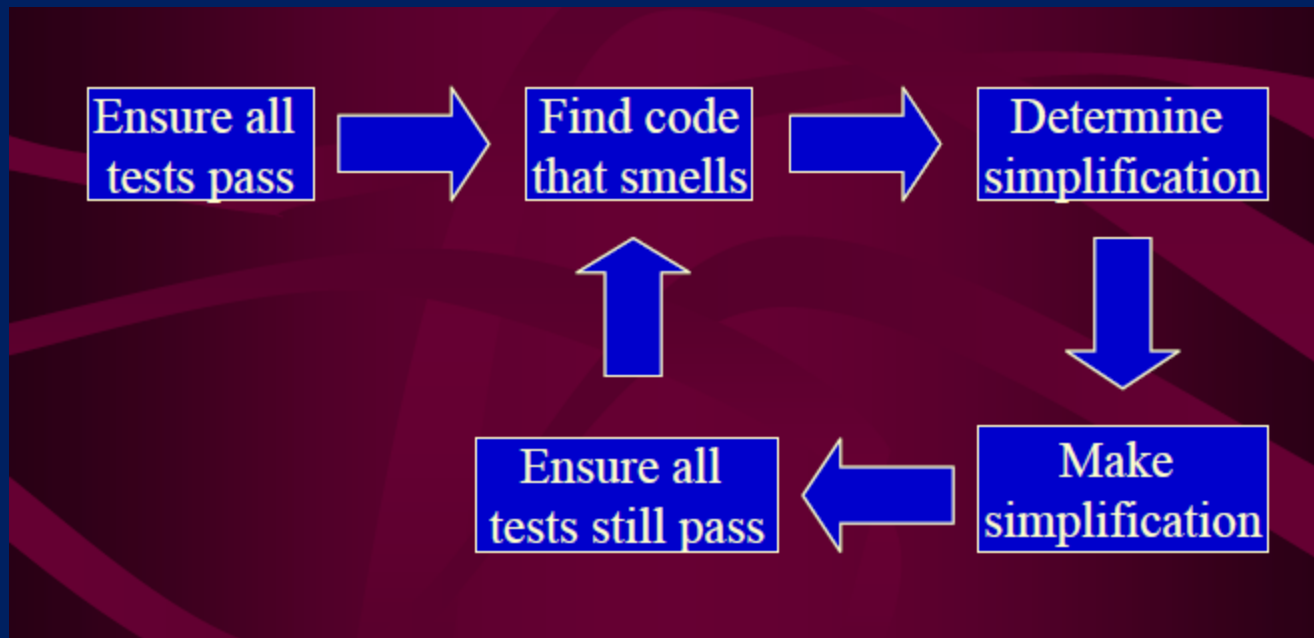
## Characteristics of Refactoring

- ▣ Source to source transformation
- ▣ Remain inside the same language, e.g., Java to Java
- ▣ Does not change the programs behaviour
- ▣ Originally designed for object-oriented languages, but can also be applied to non-object oriented language features, i.e., functions

# Refactoring

- ▣ "code smells," which means that there are indicators that point to bad code.

Refactoring has these simple steps:



# Refactoring

## Code Smells

- ▣ Indicators that something may be wrong in the code
- ▣ Can occur both in production code and test code

# Code Smells

## 1- Duplicated Code

- ▣ If you see the same code structure in more than one place, you can be sure that your program will be better if you find a way to unify them
- ▣ The simplest duplicated code problem is when you have the same expression in two methods of the same class
  - Perform *Extract Method* and invoke the code from both places
- ▣ Another common duplication problem is having the same expression in two sibling subclasses
  - Perform *Pull Up method*



# Refactoring

## *Extract Method*

- ▣ Extract Method is used when you have a code fragment that can be grouped together. That code is extracted and turned into a method whose name will clearly explain the purpose of the method.

# Extract Method

```
void printOwing(double amount)
{
    printBanner();
    //print details
    System.out.println ("name:" + _name);
    System.out.println ("amount" + amount);
}
```

# Extract Method

```
void printReceipt (double a)
{
    //print details
    System.out.println ("name:" + _n);
    System.out.println ("amount" + a);
}
```

# Extract Method

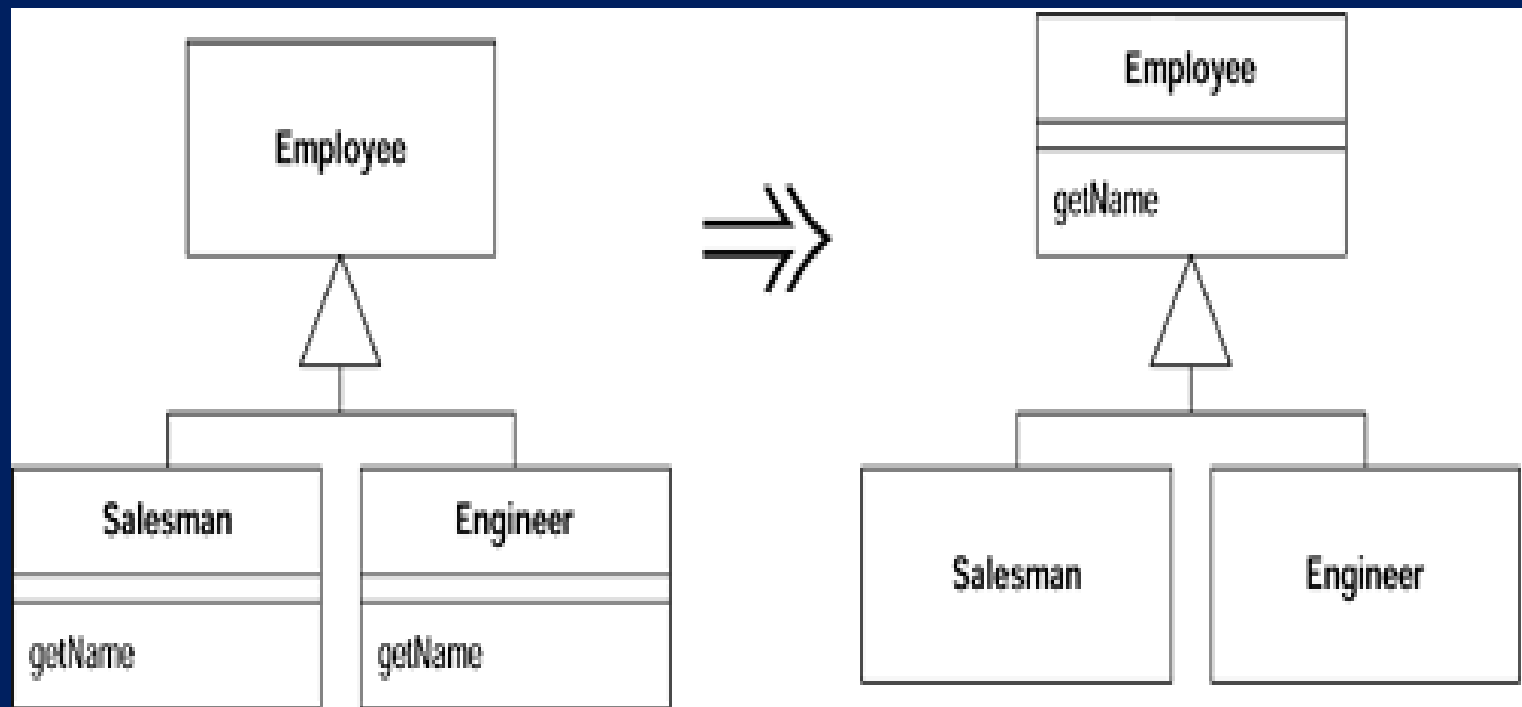
```
void printOwing(double amount)
{ printBanner();
  printDetails(amount);
}
void printReceipt (double a)
{ printBanner();
  printDetails(a);
}
void printDetails (double amount) {
  System.out.println ("name:" + _name);
  System.out.println ("amount" + amount); }
```

# Refactoring

## *Pull Up Method*

- ▣ If you find methods in different subclasses that have identical results, then you can use Pull Up Method to move them to the superclass.
- ▣ Eliminating this duplicate behavior will make the code easier to maintain and understand

# Pull Up Method



# Pull Up Method

```
1: public abstract class Vehicle
2: {
3:     // other methods
4: }
5:
6: public class Car : Vehicle
7: {
8:     public void Turn(Direction direction)
9:     {
10:         // code here
11:     }
12: }
13:
14: public class Motorcycle : Vehicle
15: {
16: }
17:
18: public enum Direction
19: {
20:     Left,
21:     Right
22: }
```

# *Pull Up Method*

```
1: public abstract class Vehicle
2: {
3:     public void Turn(Direction direction)
4:     {
5:         // code here
6:     }
7: }
8:
9: public class Car : Vehicle
10: {
11: }
12:
13: public class Motorcycle : Vehicle
14: {
15: }
16:
17: public enum Direction
18: {
19:     Left,
20:     Right
21: }
```



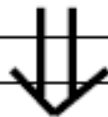
# Code Smells

## 2- Long Method

- ▣ The longer a procedure is the more difficult it is to understand
- ▣ Nearly all of the time all you have to do to shorten a method is *Extract Method*.
- ▣ As rule of thumb “more than 1 screen is too long”

# *Extract Method*

```
void printFormatted(string text) {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
    printRest(text);  
}
```



```
void printFormatted(string text) {  
    printHeader();  
    printRest(text);  
}  
  
printHeader() {  
    System.out.println("Copyright (c) 1006, Siemens AG");  
    System.out.println("Author: Michael Stal");  
}
```

# Code Smells

## 3- Large Class

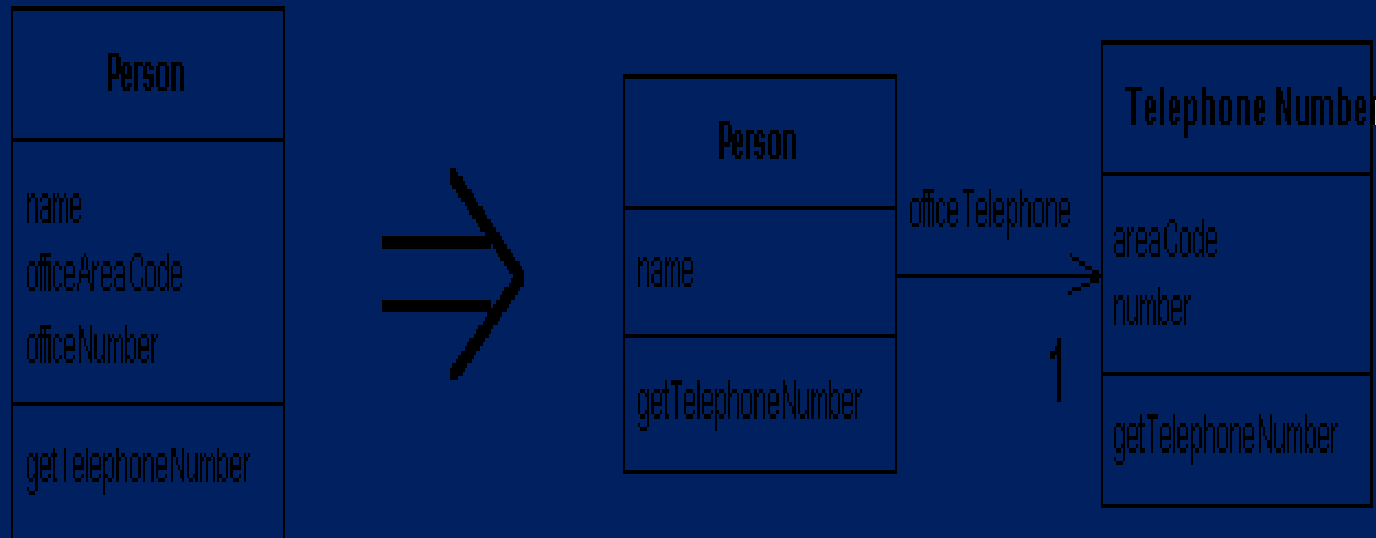
- ▣ When a class is trying to do too much, it often shows up as too many instance variables. When a class has too many instance variables, duplicated code cannot be far behind.
- ▣ A class with too much code is also a breeding ground for duplication
- ▣ In both cases *Extract Class* and *Extract Subclass* will work

# Refactoring

## *Extract Class*

- ▣ If you find that you have one class that is doing work that should really be done by two, use Extract Class. You will create a new class and move the relevant methods and attributes from the old class into the new one.

# *Extract Class*

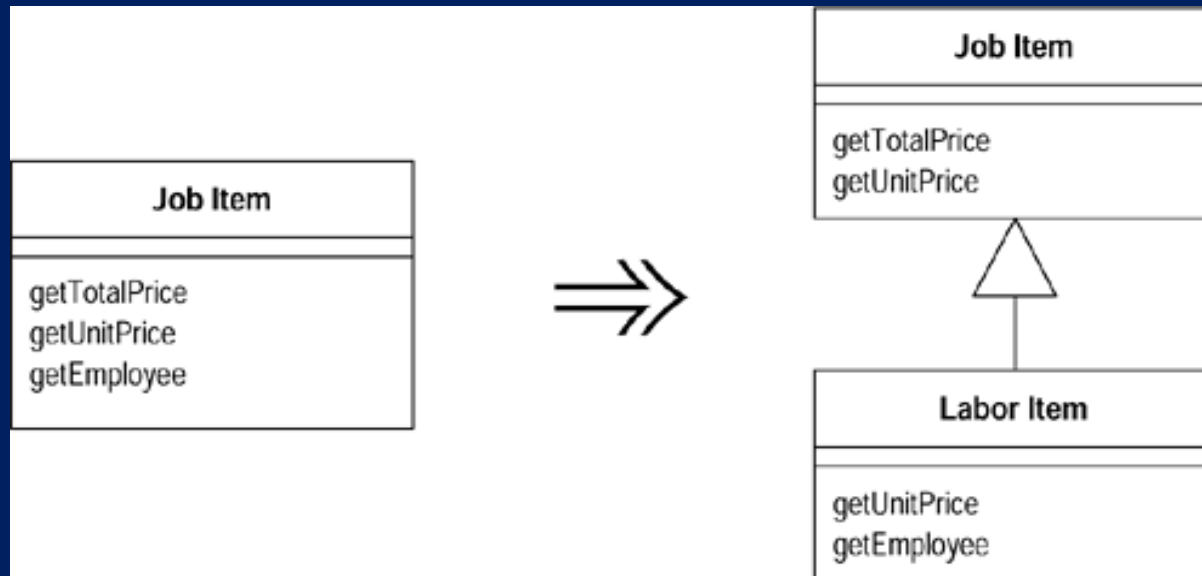


# Refactoring

## *Extract Subclass*

- ▣ Used when a class has some behavior used for some instances and not for others
- ▣ Make a subclass that inherits the common functionality

# *Extract Subclass*



# Code Smells

## 4- Long Parameter List

- ▣ Long parameter lists are hard to understand, because they are inconsistent and difficult to use because you are forever changing them as you need more data
- ▣ Use *Replace Parameter with Method* when you can get the data in one parameter by making a request of an object you already know about



# Refactoring

## Replace Parameter with Method

- ▣ An object invokes a method, then passes the result as a parameter for a method.
- ▣ The receiver can also invoke this method.

# Replace Parameter with Explicit Methods

```
void setValue (String name, int value) {  
    if (name.equals("height")) {  
        _height = value;  
        return;  
    }  
    if (name.equals("width")){  
        _width = value;  
        return;  
    }  
}
```

# Replace Parameter with Explicit Methods

```
void setHeight(int arg) {  
    _height = arg;  
}  
void setWidth (int arg) {  
    _width = arg;  
}
```

# Code Smells

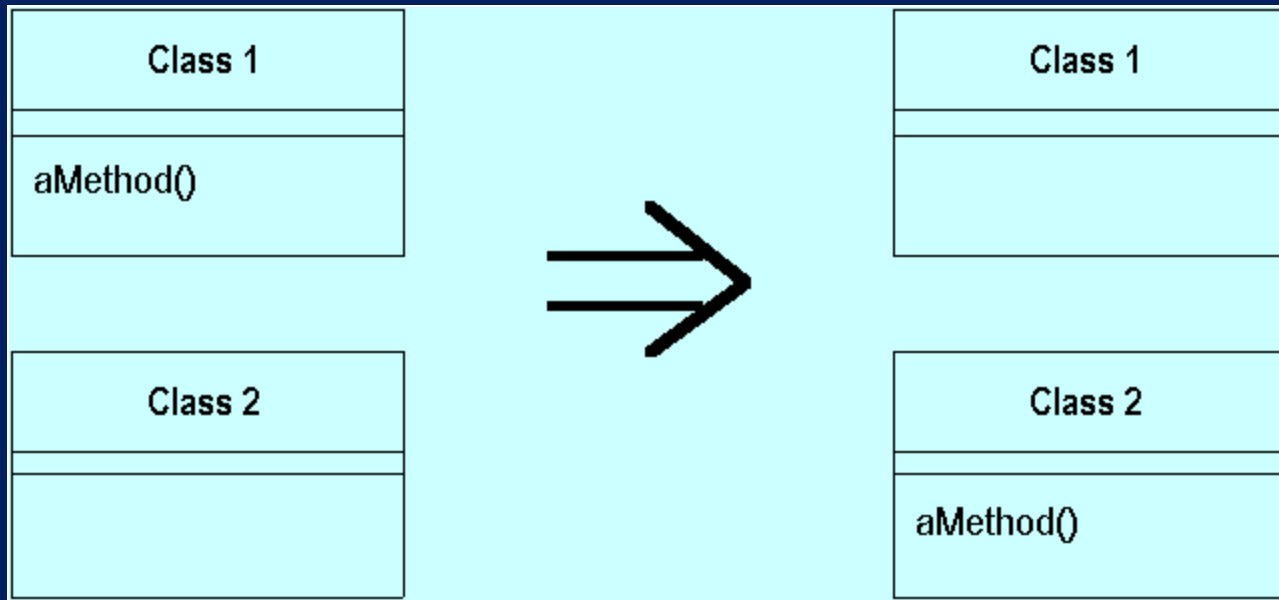
## 5- Feature Envy

- ▣ A classic smell is a method that seems more interested in a class other than the one that it is in
- ▣ For example methods that make extensive use of another class may belong in another class. Consider moving this method to the class it is so envious of.
- ▣ The method clearly wants to be elsewhere, so use *Move Method* to get it there

# Refactoring

## Move Method

Create a new method with a similar body in the class it uses most



# Move Method

```
class Project {  
    Person[ ] participants;  
}  
class Person {  
    int id;  
    boolean participate(Project p) {  
        for(int i=0; i<p.participants.length; i++)  
            { if (p.participants[i].id == id) return(true);  
              }  
        return(false);  
    }  
}  
... if (x.participate(p)) ...
```

# Move Method

```
class Project {  
    Person[ ] participants;  
    boolean participate(Person x)  
    { for(int i=0; i<participants.length; i++) {  
        if (participants[i].id == x.id) return(true);  
    } return(false);  
    } }  
class Person { int id;  
}  
... if (p.participate(x)) ...
```

# Code Smells

## 6- Switch Statements

- ▣ Most times when you see a switch statement you should consider polymorphism
- ▣ Often the switch statement switches on a type code. You want the method or class that hosts the type code value
- ▣ Use *Extract Method* to extract the switch statement and then *Move Method* to get it into the class where the polymorphism is needed. Use *Replace conditional with polymorphism*



# Refactroing

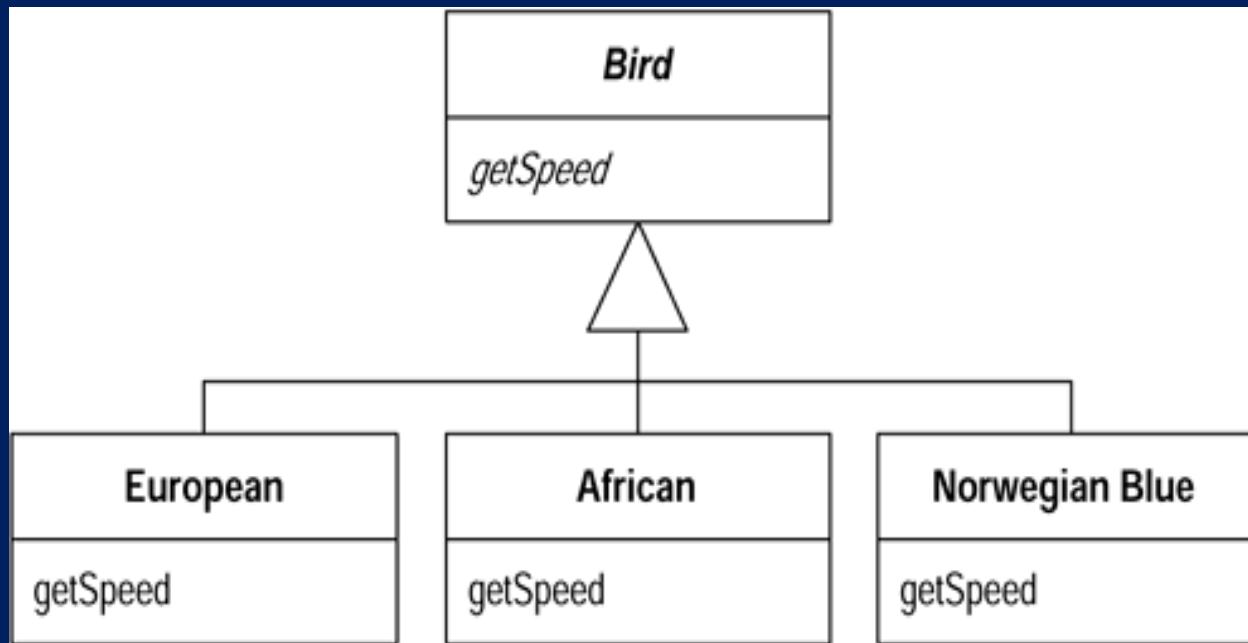
- ▣ *Replace conditional with polymorphism*

Move each leg of the conditional to an overriding method in a subclass.

# Replace conditional with polymorphism

```
Double getSpeed () {  
    switch (_type) {  
case EUROPEAN:  
    return getBaseSpeed();  
case AFRICAN:  
    return getBaseSpeed() -  
getLoadFactor() * _numCoconuts;  
case NORWEGIAN_BLUE:  
    return (_isNailed) ? 0  
: getBaseSpeed(_voltage);  
    }  
}
```

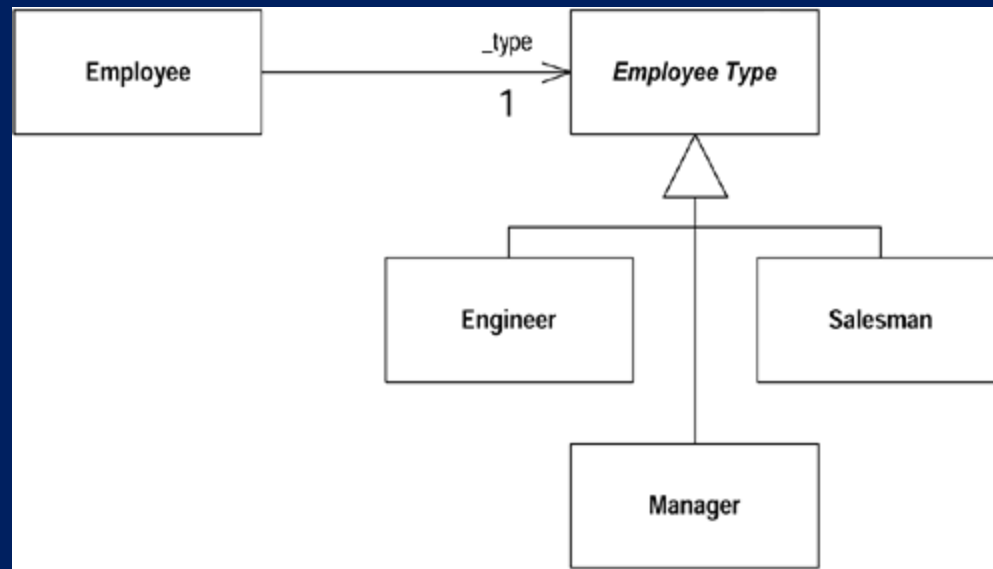
# Replace conditional with polymorphism



# Replace conditional with polymorphism

```
class EmployeeType
... int payAmount(Employee emp)
{ switch (getTypeCode())
{
    case ENGINEER:
        return emp.getMonthlySalary();
    case SALESMAN:
        return emp.getMonthlySalary() + emp.getCommission();
    case MANAGER:
        return emp.getMonthlySalary() + emp.getBonus();
    default: throw new RuntimeException("Incorrect Employee"); } }
```

# Replace conditional with polymorphism



# Replace conditional with polymorphism

class Engineer...

```
int payAmount(Employee emp) {  
    return emp.getMonthlySalary(); }  
class Salesman...
```

```
    int payAmount(Employee emp)  
    { return emp.getMonthlySalary() + emp.getCommission(); }  
class Manager...
```

```
int payAmount(Employee emp)  
    { return emp.getMonthlySalary() + emp.getBonus(); }  
and then declare the superclass method abstract:
```

class EmployeeType...

```
    abstract int payAmount(Employee emp);
```

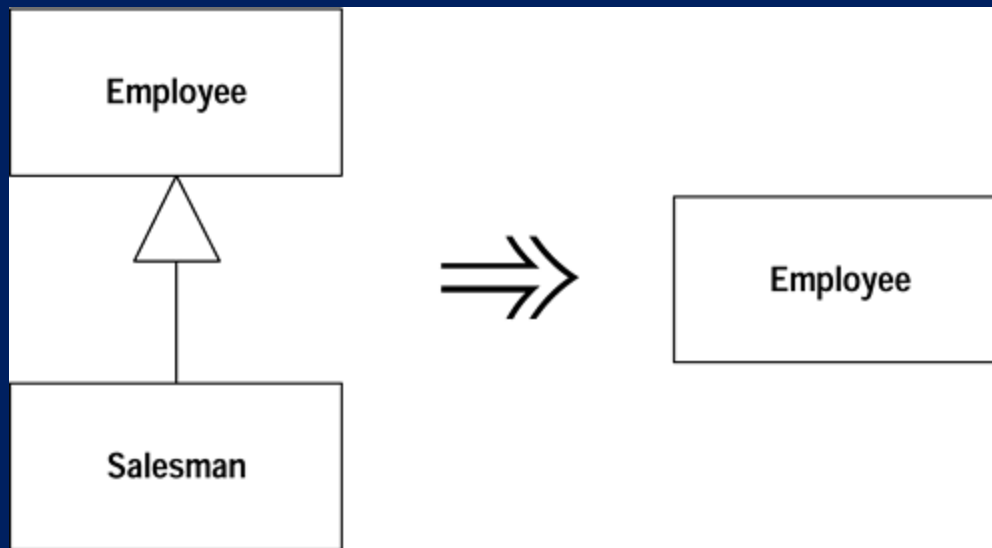
# Bad Smells

## 7- Lazy Class

- ▣ Each class you create costs money and time to maintain and understand
- ▣ A class that is not carrying its weight should be eliminated
- ▣ If you have subclasses that are not doing enough try to use *Collapse Hierarchy* and nearly useless components should be subjected to *Inline Class*

# Refactoring

*Collapse Hierarchy* A superclass and subclass are not very different.

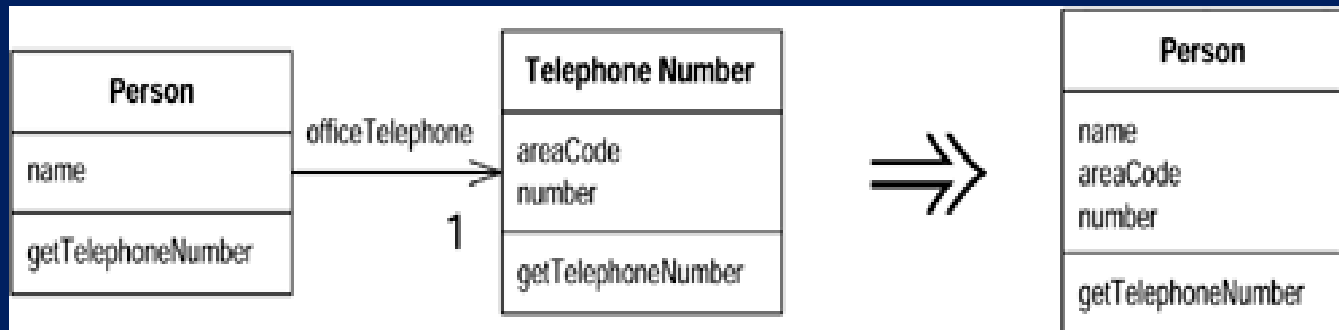




# Refactoring

## ▣ Inline Class

- ▣ *Inline Class* is the reverse of Extract Class. Use *Inline Class* if a class is no longer pulling its weight and shouldn't be around any more. Often this is the result of refactoring that moves other responsibilities out of the class so there is little left



# Bad Smells

## 8- Dead Code

Blocks of unused or commented out code in your project? Don't just leave it there just incase you need it, test to make sure you don't need it now and get rid of it. If you don't you're only going to make another developer spend a long time looking at it and scratching their head trying to work out what it is there for. Ruthlessly delete code that isn't being used.

# Bad Smells

## 9- Magic Number

### Replace Magic Number with Symbolic Constant

- ▣ Create a constant, name it after the meaning, and replace the number with it

# Refectory

```
static double ConvertPoundsToKilos( double  
    pounds ) { return pounds / 2.2 ; }
```

```
static final double  
    KILO_CONVERSION_FACTOR = 2.2;  
static double ConvertPoundsToKilos( double  
    pounds ) { return pounds /  
    KILO_CONVERSION_FACTOR ; }
```